MoodTune AI: Song Emotion Analysis and Emotion-Based Music Recommendation Using DistilBERT

Harish LG
UG Student
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India
Igharish6566@gmail.com
ORCID: 0009-0009-7739-4718

Sagadevan S
UG Student
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India
sagadevan2432006@gmail.com
ORCID: 0009-0004-6792-1856

Lakshan P
UG Student
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India
lakshanpalanikumar12@gmail.com
ORCID: 0009-0000-3226-632X

Rekhadevi B
Assistant Professor
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India
rekhadevi.se@vistas.ac.in
ORCID: 0009-0008-5253-9450

Abstract—Streaming services still lean on genre tags and collaborative filtering when they recommend music, and rarely ask what a song actually feels like. This paper describes MoodTune AI, a lightweight lyric-based emotion classifier and recommender built around a fine-tuned DistilBERT model, and investigates the challenges involved in learning a severely underrepresented emotion category. The base paper we build on, LyEmoBERT, reports 92% accuracy across four emotions (Happy, Sad, Angry, Relaxed) using a human-labeled seed set. We target six emotions: Happy, Anger, Sad, Fear, Love, and Motivational, with every label generated automatically by a pretrained classifier, since no human-annotated data was available for this project. This broader taxonomy resulted in a highly imbalanced dataset: Motivational made up under one percent of the 5,910song corpus. We tried three methods to fix the problem: training the model as-is, downsampling the majority classes, and adjusting the loss function with weights. Out of these only the loss weighting approach helped, though the improvement was small: accuracy reached 70% and the Motivational F1 score went from 0.00 to 0.42. Later we collected Motivational songs by hand and added them to the dataset, increasing the total from 44 to 75 songs. We retrained the model with this dataset and saw the Motivational F1 rise to 0.52. The trade-off was a drop in overall accuracy down to 67.6%. We include all these results, the two attempts that made things worse, because the failures taught us as much as the one method that worked.

Index Terms—Music Emotion Recognition, Music Recommendation, DistilBERT, Natural Language Processing, Class Imbalance, Transfer Learning

I. Introduction

Most music recommendations still rely on genre and what someone has listened to before. Although these approaches are effective, they may not adequately capture the emotional characteristics of a song. Someone looking for something to match a day, or something to help them get through a workout, is showing an intent that a genre label cannot really capture. Lyrics provide that information clearly, in a way that audio features usually only suggest, and yet emotion signals based on lyrics are still not used much in practice.

MoodTune AI is an effort at filling that gap. A user types in a title through a Flask web interface; the app finds the matching lyrics from a local dataset, and a fine-tuned DistilBERT model identifies the main emotion. The interface then shows a confidence score along with five other songs that have that same emotion, each linked to YouTube.

The primary focus of this work is not the model architecture itself, but the challenge of learning an emotion category that was severely underrepresented in the training data. We investigate multiple strategies for addressing this limitation and analyze both successful and unsuccessful approaches.

II. Related Work

Work on music emotion recognition, or MER, generally splits into two types: audio-based and lyric-based methods. Hu and Downie [1] were among the first to compare these two directly. They found that lyrics bring in information that audio alone does not capture. Malheiro et al. [2] followed up with a structured way of extracting features from lyrics, focusing on different mood categories. Later, Chen and Tang [3] showed that deep learning models outperform bag-of-words techniques when it comes to analyzing emotion in song lyrics. Patra et al. [4] explored multimodal recognition for Hindi and

Bengali songs. They observed that even when audio quality is low, models that use lyrics still perform well. Santana et al. [5] shifted toward using transformer-based text encoders for mood classification. More recently, Pizarro et al. [6] examined transfer learning in low-resource settings — the kind of scenario this paper focuses on.

The work we build directly on is LyEmoBERT [7]. That model fine-tunes the full BERT architecture on the Music4All dataset, which has 28,000 songs. It uses transfer learning from a set of human-labeled examples and achieves 92% accuracy on four emotion classes. Our setup differs from LyEmoBERT in three ways. First, we use six emotion classes instead of four. We include Motivational as a class, which turned out to be particularly hard to classify. Second, we generate all the labels automatically using a pretrained classifier [9] rather than starting from human-labeled seed data. Third, we use DistilBERT [8] instead of full BERT. We do this mainly because our project runs on a CPU-only laptop, and DistilBERT is lighter and faster to use. None of these choices make the accuracy numbers directly comparable, and Section V-E returns to that point.

Both BERT and its distilled variant build on the Transformer architecture [10]. BERT itself [11] established the pre-training and fine-tuning paradigm this work relies on, and DistilBERT [8] retains most of its accuracy at a fraction of the computational cost, which is why it was selected for CPU-only training. The HuggingFace Transformers library [12] was used throughout this work to load, fine-tune, and evaluate the model.

Handling severe class imbalance, as encountered here with the Motivational category, is a long-studied problem outside music emotion recognition as well. Chawla et al. [13] proposed generating synthetic minority-class examples through interpolation between existing neighbors, though this technique assumes enough minority examples exist to interpolate between in the first place — a condition not met here given only 44 original Motivational examples. Cui et al. [14] proposed reweighting losses by the effective number of samples per class rather than raw inverse frequency, a refinement of the simpler inverse-frequency weighting used in Experiment 3.

III. Dataset and Labeling Methodology

The raw dataset was sourced from a Kaggle lyric collection spanning 21 artists. Per-artist CSV files were combined into a single corpus of 6,027 songs with fields for Artist, Title, Album, Date, Lyric, and Year. During preprocessing, a hidden zerowidth-space character embedded in several song titles (e.g., “thank u, next”) was identified and removed, since it silently broke exact-match title lookups used by the recommendation lookup logic.

No emotion labels existed in the source data. Labels were therefore generated automatically using a pretrained emotionclassification model (based on the dair-ai/emotion label set, which includes a genuine “love” category [9]). The classifier’s sixth native output category, “surprise,” was remapped to Motivational as the closest available substitute, since no pretrained classifier with a genuine motivational category was available; Section VI revisits this substitution as a likely source of label noise in that class specifically. To reduce bias from lyrics whose opening lines are not representative of the whole song, a multichunk sampling strategy was used: text was sampled from the beginning, middle, and end of each song’s lyrics, classified independently, and combined via majority vote. This produced an initial labeled set of 6,000 songs, obtained by dropping records with missing lyrics and retaining the first 6,000 remaining rows. Of these, 71 songs with explicit or profane titles were subsequently removed from the dataset entirely, leaving 5,910 songs. This 5,910-song set, summarized in Table 1, is the actual dataset used for training and for song recommendations alike across the experiments in Section V.

Table 1. Class Distribution of the 5,910-song Labeled Dataset

Emotion Class	Count	Percentage
Happy	2,329	39.4%
Anger	1,821	30.8%
Sad	1,058	17.9%
Fear	344	5.8%
Love	314	5.3%
Motivational	44	0.7%

Table 1 illustrates the severe class imbalance inherent to the automatically labeled dataset, also visualized in Fig. 1. The Motivational class, with only 44 examples (0.7%), is roughly 53 times smaller than the majority Happy class, and constitutes the central technical challenge addressed in Section V.

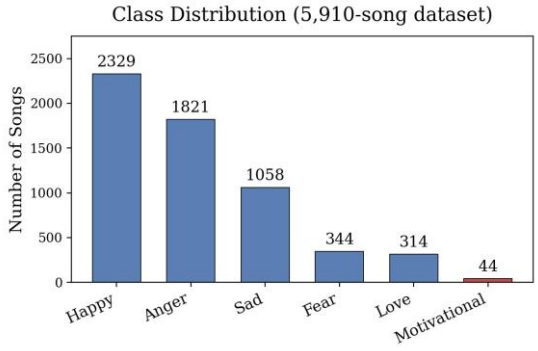


Figure 1. Class distribution across the six emotion categories, showing the severe scarcity of the Motivational class relative to the others.

IV. Proposed Methodology

The classification model is a fine-tuned distilbert-base-uncased model, trained using the HuggingFace Transformers library on

a CPU-only development machine. The labeled dataset was split into training and test subsets using an 80/20 stratified split with a fixed random seed (42) to preserve class proportions across both subsets. Lyrics are tokenized using the DistilBERT tokenizer and truncated to a maximum sequence length of 256 tokens, and the model is fine-tuned for sequence classification across the six target emotion classes. Training used a batch size of 8 with the AdamW optimizer (the HuggingFace Trainer default). All experiments in this paper were trained for three epochs unless otherwise noted. Experiments 1–3 used the HuggingFace Trainer's default learning rate of 5×10^{-5} with linear decay, while Experiment 4 used a smaller learning rate of 2×10^{-5}, as described in Section V-D.

To address class imbalance, a custom WeightedTrainer class was implemented, subclassing the HuggingFace Trainer and overriding the loss computation to apply per-class weights within the cross-entropy loss function, computed as the inverse of each class's relative frequency in the training set.

At inference time, the Flask application accepts a song title, retrieves the corresponding lyric text from the local dataset, and passes it through the fine-tuned model. The softmax output is displayed to the user as a confidence percentage alongside the predicted emotion label, and five additional songs sharing the predicted emotion are sampled (rather than deterministically selected) from the recommendation pool to provide variety across repeated queries for the same emotion.

V. Experimental Results

We ran four training experiments in total, in the order described below, each one a reaction to what the previous attempt got wrong. Results are summarized in Tables 2 and 3.

A. Experiment 1 — Unweighted Baseline

The first attempt was the obvious one: train on the full 5,910song dataset with ordinary cross-entropy loss, no adjustments, for three epochs. Overall accuracy came out at 72%, and five of the six classes learned reasonably well — F1-scores of 0.80 for Happy, 0.72 for Anger, 0.68 for Sad, 0.61 for Fear, 0.49 for Love. Motivational scored 0.00. With only 44 examples to learn from, the model never once predicted it correctly on the held-out test set; it simply defaulted to a majority class every time.

B. Experiment 2 — Class Balancing via Resampling

The next idea was to rebalance the classes by capping every majority class at 350 examples, shrinking the dataset down to 1,755 songs, and training again for three epochs. This made things worse across the board — accuracy dropped to 51%, and Motivational still scored 0.00. The reasoning in hindsight is straightforward: resampling cannot increase the diversity of the minority class when only a very small number of original examples are available. Cutting the majority classes down just threw away most of the useful training signal without giving the minority class anything new to learn from. This told us the real problem was shortage, not a simple imbalance, and we stopped using the resampling method.

C. Experiment 3 — Class-Weighted Loss

This experiment kept the 5,910-song dataset exactly as it was; rather than changing the data, it changed the loss function, applying a penalty to mistakes on rare classes through a custom WeightedTrainer. The weights were set by inverse class frequency: 0.11 for Anger, 0.58 for Fear, 0.09 for Happy, 0.64 for Love, 0.19 for Sad, and 4.39 for Motivational. That means Motivational had forty-nine times the weight of Happy. Following the application of these class weights, the Motivational F1-score increased from zero to 0.42. Precision for Motivational was 0.30, recall was 0.70. So the model now caught seven out of ten truly motivational songs and the overall accuracy dropped only by two points to seventy percent. That version was the one that really worked and it became the deployed model, checkpoint1794, for the rest of the project, until Experiment 4.

D. Experiment 4 — Dataset Expansion (Final DeployedModel)

After Experiment 3 was working a simpler fix was still possible: to add manually identified Motivational examples instead of just using loss weighting to make up for little data. 31 additional songs manually identified as belonging to the Motivational category were sourced, including well-known anthem songs such as Eye of the Tiger, Till I Collapse, and We Are the Champions, and added to the dataset as new rows. This increased the Motivational class from 44 songs to 75 songs — a 70.5% rise.

We retrained with the dataset for five epochs using a slightly smaller learning rate of 2×10^{-5} and the same inversefrequency weighting approach, which was recalculated automatically for the new numbers of each class. The result was an improvement on the key metric: Motivational F1 increased from 0.42 to 0.52 while overall accuracy settled at 67.6%. This is a little lower than Experiment 3's 70%. It is a real gain in the Motivational category, the area I was trying to fix. Fig. 2 summarizes this trade-off across all four experiments, and Fig. 3 breaks the per-class effect down further. This model was verified working correctly in the live Flask application across all six emotion categories and is the version currently deployed.

It is worth noting that an intermediate attempt at automatically selecting the "best" training epoch (rather than simply keeping the final one) was also tried, first by the highest overall accuracy and then by macro-averaged F1 across all classes. Both attempts either picked an epoch that had quietly regressed on Motivational specifically, or failed outright due to a metricnaming mismatch in the training library. Given time constraints ahead of project review, we did not pursue this further and instead kept the straightforward five-epoch run described above, which is simpler to reproduce and explain.

Table 2. Summary Comparison of All Four Training Experiments

Experiment	Accuracy	Motiv. F1
1. Baseline (Unweighted)	72%	0.00
2. Resampling (Rejected)	51%	0.00
3. Class-Weighted Loss	70%	0.42
4. Dataset Expansion (Final)	67.6%	0.52

Table 3. Per-class F1-score Comparison Across Experiments (N/R = Not Reported)

Class	Exp.1	Exp.2	Exp.3	Exp.4
Happy	0.80	N/R	0.76	0.74
Anger	0.72	N/R	0.70	0.66
Sad	0.68	N/R	0.66	0.66
Fear	0.61	N/R	0.64	0.60
Love	0.49	N/R	0.55	0.50
Motivational	0.00	0.00	0.42	0.52

Per-class scores for Experiment 2 beyond Motivational were not carried forward, since resampling was rejected outright once its overall accuracy and Motivational performance were both found to be worse than the baseline; there was little point in a fuller breakdown of an approach we had already ruled out.

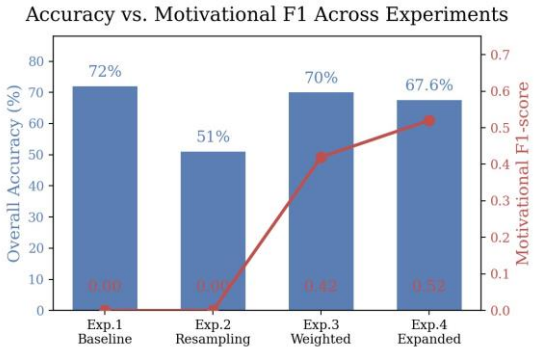


Figure 2. Overall accuracy (bars) against Motivational F1-score (line) across all four experiments. Motivational F1 improves monotonically from Experiment 2 onward, at a small and non-monotonic cost to overall accuracy.

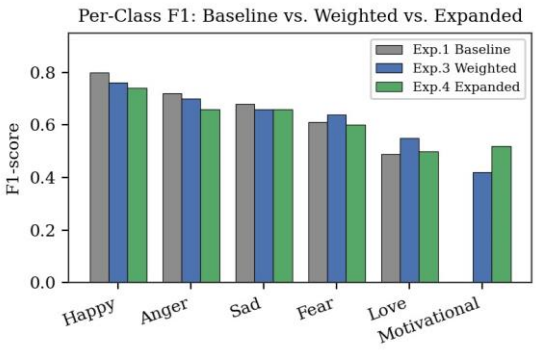


Figure 3. Per-class F1-score for Experiments 1, 3, and 4.

Motivational is the only class that improves substantially across the three; most other classes drift down slightly by Experiment 4.

E. Comparison with Base Paper

Table 4. Comparison Between LyEmoBERT and MoodTune AI

Metric	LyEmoBERT [7]	MoodTune AI
Accuracy	92%	67.6%
Emotion Classes	4	6
Label Source	Human-seeded	Automated
Base Model	Full BERT	DistilBERT
Dataset Size	28,000	5,941

The comparison shows a gap between LyEmoBERT's 92 percent and MoodTune AI's 67.6 percent, as summarized in Table 4. Three things explain most of that gap. First, LyEmoBERT looks at four emotions while MoodTune AI looks at six. That makes the task harder for MoodTune AI because more options mean chances for mistakes. Second, the labels for MoodTune AI are produced by a system, not by human experts. So some of the ground truth is noisy. A model cannot do better than the quality of its labels. Third, MoodTune AI uses a distilled model that can be trained on a simple laptop instead of a full BERT model that needs special hardware. Therefore, the reported accuracy values should not be interpreted as a direct performance comparison, because the two studies differ in emotion taxonomy, labeling methodology, dataset size, and model architecture. A useful comparison is with our earlier experiments shown in Table 2, where the improvement, in the Motivational category, is clear.

VI. Discussion and Limitations

Before settling on Experiment 4 we also tried a careful fix on the original 44-song Motivational set. We manually went through each song, correcting the obvious mislabels. Out of the 44 songs, 39 were actually mislabeled by the automated classifier. Songs like "Crazy in Love" and "Teenage Dream" had been tagged Motivational even though they had nothing to do with motivation. This observation highlights a key limitation of automatically generated emotion labels: errors in the labeling model can propagate into the training dataset and directly affect downstream classification performance. The severity of this particular mislabeling rate is consistent with the category's origin described in Section III: since the classifier had no genuine motivational output and Motivational was instead constructed by remapping its native "surprise" predictions, many songs the classifier judged as merely surprising rather than motivational would be expected to land in this category incorrectly. After fixing the labels, five songs truly belonged in the Motivational category. When we retrained the model on this cleaned but smaller set, overall accuracy improved to 72% and Love's F1 score went up to 0.59. Motivational's F1 score

dropped to 0.00. With five examples, any 80/20 train-test split would leave just four for training and one for testing. The extremely small number of correctly labeled examples provides insufficient evidence for reliable generalization. A live test made the regression concrete: a song that had been correctly classified as Motivational reverted to a 99.44%-confidence Happy prediction after this retrain. We reverted the labels and kept checkpoint-1794 as the working model at that point, which is why Experiment 4 ultimately took the opposite approach — adding manually identified examples rather than cleaning the existing few.

A handful of songs that were briefly relabeled during that rejected cleanup attempt ("English Rose," "Crazy in Love," "MY POWER," and "perfect" among them) can still show up mispredicted in the live app relative to what a listener might intuitively expect, since the deployed model was trained on the original automated labels for those specific titles rather than the manually corrected ones. We consider this a minor, well-understood inconsistency rather than a bug.

The two experiments that did not work — resampling and label cleaning — both lead to the same conclusion. Motivational's problem wasn't really about the training strategy. It was simply that there weren't enough reliably labeled examples to work with. Loss weighting (Experiment 3) got partway there by making the existing 44 examples count for more; actually going out and finding 31 more manually identified examples (Experiment 4) is what moved the needle further.

The accuracy dip between Experiment 3 and Experiment 4 (70% down to 67.6%) is worth being honest about rather than glossing over. Looking at the confusion matrix for the final model (Fig. 4), most of that drop traces back to Anger being confused with Happy and Sad — a pattern that predates Experiment 4 and is unrelated to the Motivational-category changes. This suggests a further, separate opportunity: reviewing a sample of the Anger-labeled songs most often confused with other classes, since some of these may themselves be automatedlabeler mistakes rather than genuine model weaknesses. We did not have time to pursue this before the current review cycle, but note it here as the most promising direction for anyone continuing this work.

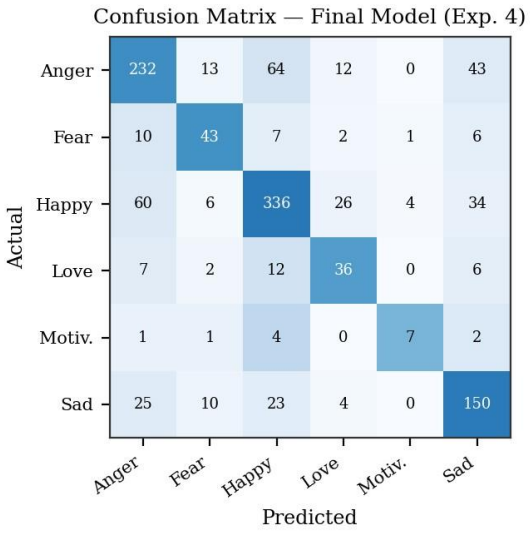


Figure 4. Confusion matrix for the final deployed model (Experiment 4). Anger is most often confused with Happy (64 songs) and Sad (43 songs), while Motivational, despite its small support, is diagonally dominant.

VII. Conclusion and Future Work

This paper described MoodTune AI, a DistilBERT-based lyric emotion classifier and recommender, and walked through four training experiments aimed at one specific problem: a Motivational emotion category that started out with almost no training data. An unweighted baseline could not learn the category at all (F1 = 0.00). Resampling the majority classes down made everything worse, which told us the real issue was scarcity rather than imbalance. Weighting the loss function helped improve the model's performance on Motivational, bringing the F1 score up to 0.42 though with a drop in overall accuracy. Then, after adding 31 manually identified Motivational songs — once we had time to collect them — the F1 score rose further to 0.52. This came with an additional decrease in overall accuracy, mainly due to confusion between Anger, Happy and Sad labels. That confusion wasn't something we could fix during this round.

The model obtained from Experiment 4 was integrated into the Flask application and evaluated across all six emotion categories. For improvements two paths stand out. First, we should keep growing the dataset — 75 songs is still a small number by most standards. Second, we need to look at the Anger-labeled songs that get mixed up with Happy and Sad. Our confusionmatrix analysis suggests this might offer more gains than more work focused just on Motivational. However, the use of automatically generated labels remains an important limitation of the current study, particularly for Motivational, which was constructed by remapping a substitute category from the underlying classifier rather than a genuine model output; a classifier with a true motivational category, or

human-verified labels, would likely reduce this noise substantially.

Acknowledgment

The authors thank Guide Rekhadevi B and the Department of Computer Science and Engineering (AI & ML), VISTAS, Chennai, for guidance and support throughout this project.

References

[1] X. Hu and J. S. Downie, "Improving mood classification in music digital libraries by combining lyrics and audio," in *Proc. 10th Annual Joint Conference on Digital Libraries*, 2010.

[2] R. Malheiro, R. Panda, P. Gomes, and R. P. Paiva, "Emotionally-relevant features for classification and regression of music lyrics," *IEEE Transactions on Affective Computing*, 2013.

[3] Y. Chen and X. Tang, "Deep learning for lyric-based music emotion and sentiment classification," in *Proc. Int. Conf. on Machine Learning and Applications*, 2018.

[4] B. G. Patra, D. Das, and S. Bandyopadhyay, "Multimodal mood classification of Hindi and Bengali songs using lyrics and audio," in *Proc. Int. Conf. on Natural Language Processing*, 2017.

[5] B. S. Santana et al., "Transformer-based approaches for music mood classification from lyrics," in *Proc. Brazilian Symposium on Computer Music*, 2020.

[6] L. Pizarro et al., "Transfer learning strategies for lowresource music emotion recognition," in *Proc. Int. Society for Music Information Retrieval Conference*, 2024.

[7] S. Revathy et al., "LyEmoBERT: Lyrics-based music emotion recognition using BERT and transfer learning," *Procedia Computer Science*, vol. 218, pp. 1196–1208, 2023.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[9] E. Saravia et al., "CARER: Contextualized affect representations for emotion recognition," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACLHLT)*, 2019, pp. 4171–4186.

[12] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020, pp. 38–45.

[13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[14] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Classbalanced loss based on effective number of samples," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9260–9269.